

Almost Shortest Path

Quase Menor Caminho (UVa 12144)

Uma abordagem estruturada combinando buscas baseadas no algoritmo de Dijkstra com filtragem inteligente de arestas.

Isadora F. N. Rios

João Victor L. S. Leão

Cauan G. S. Barbosa

? O Problema Proposto

Como conectar pontos sem utilizar arestas que compõem caminhos mínimos.

Definição do Quase Menor Caminho

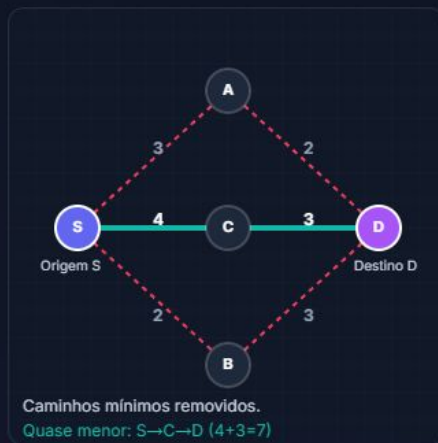
Encontrar o menor caminho entre a origem **S** e o destino **D** no grafo, desconsiderando qualquer aresta pertencente a **pelo menos um** dos caminhos mais curtos.

Regras e Casos de Borda

Se nenhum caminho alternativo restar após a exclusão de caminhos ótimos, ou se não houver conexão inicial ligando **S** a **D**, a resposta impressa deve ser **-1**.

Pesos das arestas são positivos e o grafo é direcionado.

Caminho Ótimo vs Alternativo



Caminhos Mínimos

Quase Menor

⚠ O Desafio da Escala

Por que implementações ingênuas de fila de prioridade falham em Python.

Restrições Estritas (UVa Judge)

Limite de Tempo

2.0s - 3.0s

Vértices (V)

≤ 500

Arestas (E)

≤ 10.000

Casos de Teste

Até 100+

O Problema de Fila Não Otimizada

Dijkstra executado múltiplas vezes por caso de teste exige uma fila de prioridades extremamente veloz. Abordagens sem heaps geram sobrecarga temporal massiva:

- ✗ **Busca linear na fila:** $O(V^2)$ por Dijkstra, levando a Time Limit Exceeded (TLE).
- ✗ **Cópia física de grafos:** Consome CPU e RAM de maneira ineficiente.

Tempo de Execução Acumulado

Busca Linear (Sem Heap)

> 3.5s (TLE)

Complexidade $O(V^2)$

Nossa Abordagem (Fila baseada em Heap)

~0.18s

⚡ Otimização assintótica e uso de `heapq` nativo

A Estratégia de 3 Dijkstras

Como remover arestas pertencentes a caminhos ótimos sem recriar grafos.

1

Dijkstra de Ida

Calcula as menores distâncias a partir de **S** no grafo original. Obtemos o vetor $distS$.

2

Dijkstra Reverso

Calcula as menores distâncias até **D** usando o grafo transposto (invertido). Obtemos o vetor $distD$.

3

Filtro de Arestas

Arestas (u, v) de peso W que participam de caminhos mínimos satisfazem a condição de exclusão.

4

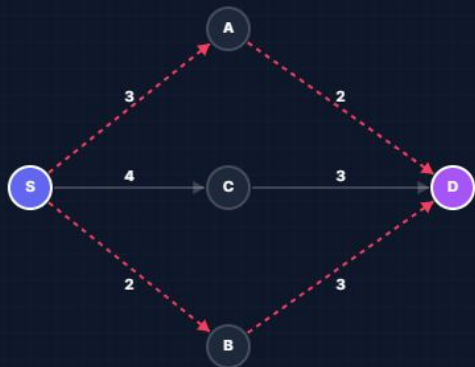
Cálculo Final

Roda o terceiro Dijkstra de **S** para **D** ignorando as arestas excluídas.

Condição de remoção: $distS[u] + peso(u, v) + distD[v] == distS[destino]$

▶ Simulador Interativo

Veja o algoritmo em execução passo a passo no grafo de exemplo.



1. Rodar Dijkstras & Filtrar

2. Quase Menor Caminho

Resetar

Console de Execução

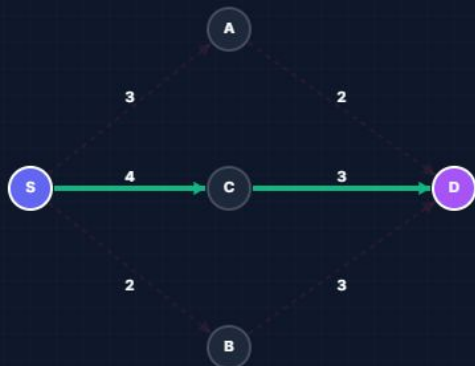
```
Simulador pronto. Grafo com 5 vertices e 6 arestas carregado.  
Rodando Dijkstra de Ida (origem S)...  
Distâncias a partir de S: S=0, A=3, B=2, C=4, D=5  
Rodando Dijkstra Reverso (destino D)...  
Distâncias até D: S=5, A=2, B=3, C=3, D=0  
Filtrando arestas ótimas via condição de menor caminho:  
- Aresta S->A (3):  $0 + 3 + 2 = 5 == 5$  (Remover!)  
- Aresta A->D (2):  $3 + 2 + 0 = 5 == 5$  (Remover!)  
- Aresta S->B (2):  $0 + 2 + 3 = 5 == 5$  (Remover!)  
- Aresta B->D (3):  $2 + 3 + 0 = 5 == 5$  (Remover!)  
- Aresta S->C (4):  $0 + 4 + 3 = 7 != 5$  (Manter)  
- Aresta C->D (3):  $4 + 3 + 0 = 7 != 5$  (Manter)  
Arestas ótimas marcadas como vermelhas e desabilitadas.
```

Legenda:

Original Removido Quase Menor

▶ Simulador Interativo

Veja o algoritmo em execução passo a passo no grafo de exemplo.



1. Rodar Dijkstras & Filtrar

2. Quase Menor Caminho

🔄 Resetar

Console de Execução

Distâncias até D: S=5, A=2, B=3, C=3, D=0

Filtrando arestas ótimas via condição de menor caminho:

- Aresta S->A (3): $0 + 3 + 2 = 5 == 5$ (Remover!)
- Aresta A->D (2): $3 + 2 + 0 = 5 == 5$ (Remover!)
- Aresta S->B (2): $0 + 2 + 3 = 5 == 5$ (Remover!)
- Aresta B->D (3): $2 + 3 + 0 = 5 == 5$ (Remover!)
- Aresta S->C (4): $0 + 4 + 3 = 7 != 5$ (Manter)
- Aresta C->D (3): $4 + 3 + 0 = 7 != 5$ (Manter)

Arestas ótimas marcadas como vermelhas e desabilitadas.

Rodando terceiro Dijkstra no grafo filtrado...

Caminho encontrado: S -> C -> D

Custo acumulado do Quase Menor Caminho: 7

Algoritmo concluído com sucesso!

Legenda:

■ Original ■ Removido ■ Quase Menor

</> Arquitetura Modular do Código

Como as responsabilidades foram separadas seguindo padrões elegantes.

`directed_edge.py / bag.py`



Aresta Direcionada e Bag: Estruturas orientadas a objetos inspiradas no algs4 para modelar arestas com peso e listas de adjacência.

`dijkstra.py`



Busca Dijkstra Otimizada: Algoritmo de busca desacoplado utilizando fila de prioridades acoplada ao heapq nativo do Python (execução $O(E \log V)$).

`edge_weighted_digraph.py`



Grafo Direcionado: Representação por dígrafo ponderado e suporte ao método `reverse()` para transpor o grafo de maneira super eficiente.

`solver.py / main.py`



Filtro e Execução: Lógica central de negócio executando os Dijkstras e filtrando as frentes ótimas, com main gerenciando I/O rápido.

Complexidade e Resultados

Análise de eficiência assintótica e validação no juiz online.

Análise Assintótica

- **Complexidade de Tempo:** Três Dijkstras independentes operando em $O((V + E) \log V)$ no pior caso.
- **Complexidade de Espaço:** Dígrafos original e reverso mapeados em memória, consumindo apenas $O(V + E)$ (menos de 5MB de RAM).
- **Uso do heapq nativo:** Escrito em C puro no interpretador oficial CPython, eliminando qualquer overhead de objetos Python na fila.

✓ **Submissão Aceita!**



Comprovante oficial de aceite (accepted.jpeg) disponível na pasta evidencias.